

Dir-GD: Directed Graph Distillation

Ce Yang

xiaoyang798@snnu.edu.cn
Shaanxi Normal University
School of Artificial Intelligence and
Computer Science
Xi'an, China

Jianrui Chen

jianrui_chen@snnu.edu.cn
Shaanxi Normal University
School of Artificial Intelligence and
Computer Science
Xi'an, China

Fei Hao*

fhao@snnu.edu.cn
Shaanxi Normal University
School of Artificial Intelligence and
Computer Science
Xi'an, China

Jia Hu

j.hu@exeter.ac.uk
University of Exeter
Department of Computer Science
Exeter, UK

Jie Gao

jg911@exeter.ac.uk
University of Exeter
Department of Computer Science
Exeter, UK

Geyong Min

g.min@exeter.ac.uk
University of Exeter
Department of Computer Science
Exeter, UK

Abstract

Graph-structured data effectively captures complex relationships in diverse domains such as social networks, financial transactions, citation networks, and recommendation systems. Graph Neural Networks (GNNs) excel in learning intricate topological patterns, yielding strong performance on tasks like node classification and link prediction. However, real-world graphs often scale to millions of nodes and billions of directed edges, posing significant computational and storage challenges for GNN training that frequently exceed available hardware limits. Although graph sampling and distillation techniques alleviate these issues by subsampling or creating surrogate graphs, they primarily handle undirected graphs, neglecting directional semantics that are crucial for applications like fraud detection and causal analysis. To address these limitations, we introduce the Directed Graph Distillation (Dir-GD) framework, which combines distributed learning with community detection to divide large directed graphs into independent subgraphs for distributed directed GNN training. This process culminates in parameter aggregation to produce a compact global synthetic graph that preserves essential topology and directionality. Extensive experiments on large-scale datasets, such as the million-node soc-pokec-relationships, demonstrate over 91% accuracy at 0.001 distillation ratios, accompanied by substantial memory and runtime savings. This work pioneers directed graph distillation as a key paradigm for analyzing ultra-large directed graphs, offering a scalable solution that maintains high fidelity in compressed representations.

CCS Concepts

• **Computing methodologies** → **Neural networks.**

*Corresponding author



This work is licensed under a Creative Commons Attribution 4.0 International License.
WWW '26, Dubai, United Arab Emirates
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2307-0/2026/04
<https://doi.org/10.1145/3774904.3792162>

Keywords

Directed Graphs, Directionality Modeling, Directed Graph Distillation (Dir-GD), Distributed Learning, Graph Distillation

ACM Reference Format:

Ce Yang, Fei Hao, Jie Gao, Jianrui Chen, Jia Hu, and Geyong Min. 2026. Dir-GD: Directed Graph Distillation. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 10 pages. <https://doi.org/10.1145/3774904.3792162>

1 Introduction

Graph-structured data serves as a foundational representational paradigm in a wide array of domains, including social networks [40], academic citation networks [19], recommendation systems [7], and chemical molecule prediction [28]. These graphs, often manifesting as large-scale directed structures with millions of nodes and billions of edges, encode complex entity interactions and dynamic topological relationships, providing rich semantic foundations for downstream tasks such as node classification, link prediction, graph classification, and community detection.

Graph Neural Networks (GNNs) [6, 18, 32] employ recursive message passing and aggregation mechanisms to learn node embeddings that capture both local neighborhood information and global structural dependencies, resulting in strong empirical performance across various graph-related tasks. However, the rapid exponential growth in graph scales introduces substantial challenges to traditional GNN training pipelines: the inherent interdependence among neighboring nodes escalates computational complexity in proportion to node degrees and graph densities, leading to inflated memory requirements and communication overheads. In distributed computing environments or under resource-constrained settings, such as edge devices, loading the entire graph into memory incurs high latencies, potential out-of-memory errors, and hardware limitations. Moreover, the heterogeneity of graph structures (encompassing varying node degrees, feature distributions, and edge directions) further complicates centralized processing approaches, highlighting the urgent need for more efficient graph compression techniques and distributed learning paradigms. A pivotal research question emerges: How can we effectively compress ultra-large directed graphs while faithfully preserving their core topological features

and semantic properties, thereby accelerating GNN training and inference processes, reducing storage footprints, and enhancing overall task efficiency in practical deployments?

Existing efforts have attempted to tackle these challenges through a variety of methods, including graph sampling [38], graph distillation [17], and distributed learning frameworks [12]. Graph sampling techniques [5] utilize random or biased strategies, such as node sampling, edge sampling, or random walk-based subsampling, to reduce the computational load while striving to retain key statistical properties of the original graph; nevertheless, aggressive compression ratios often lead to the inadvertent loss of critical structural information or semantic details, consequently degrading the performance of downstream models. Graph distillation methods, on the other hand, aim to synthesize smaller surrogate graphs that are optimized to match the performance of GNNs trained on the full graph, thereby facilitating efficient compression and knowledge transfer across models. However, these approaches predominantly target undirected graphs, thereby ignoring the rich semantics embedded in directed edges. Furthermore, the majority of distillation frameworks rely on teacher-student architectures, where a student model is trained to mimic a pre-trained teacher model on the full graph, but this paradigm does not directly address the resource limitations during the distillation phase itself, often requiring substantial computational resources for the initial teacher training. Distributed learning extensions, inspired by federated learning principles, help mitigate computational silos in graph data by enabling collaborative training across multiple nodes, but they typically assume access to the complete graph structure at some stage, making them ill-suited for truly distributed scenarios with communication constraints, privacy concerns, or data silos. These collective limitations significantly restrict the applicability of prior methods in direction-sensitive tasks, such as fraud detection in financial networks, where edge directions indicate transaction flows, or causal analysis in biological pathways, where preserving orientation is essential for accurate inference.

To bridge these identified gaps in the literature, we propose the Directed Graph Distillation (Dir-GD) framework, the first distributed distillation methodology explicitly designed for large-scale directed graphs. Dir-GD seamlessly integrates distributed learning principles with directed graph distillation techniques. It begins by employing Infomap community detection to partition the original graph into autonomous subgraphs that capture inherent modular structures and directional information flows, analogous to the natural division of China into provinces where cities within the same province (e.g., Guangdong or Sichuan) exhibit far denser internal connections, such as frequent intra-provincial high-speed rail trips, local trade cycles, and population movements, relative to cross-provincial links. Parallel training of directed GNNs is then conducted on each subgraph, eliminating the need to load the full graph into memory and mirroring how each province could independently develop a local model focused on its distinctive regional flows (for instance, Guangdong emphasizing export-oriented manufacturing linkages while Sichuan prioritizes inland migration patterns), thereby enabling truly distributed computation without centralizing sensitive nationwide data. Within each community, Dir-GD selects central nodes via directed PageRank, comparable to identifying the most influential provincial capitals (e.g., Beijing,

which dominates inbound policy flows and outbound economic influence, or Shanghai as a financial hub with disproportionate outgoing investment edges), and applies edge-count-based weighting to preserve directed asymmetries (e.g., reflecting that capital or passenger flows from Beijing to peripheral cities greatly exceed reverse directions, thus avoiding the loss of real-world causal semantics inherent in symmetrization approaches). This ensures high-fidelity representations even at extreme compression ratios, as illustrated in Fig. 1. Finally, learnable parameter aggregation fuses knowledge from these subgraphs into a compact global synthetic graph, akin to synthesizing a comprehensive national overview from specialized provincial insights, where each "provincial expert" contributes parameterized knowledge of local dynamics to yield a miniature yet expressive "national map" that captures nationwide patterns without requiring access to the raw, full-scale urban network. In this architectural design, local subgraph models function as specialized domain experts imparting insights from their respective modular components, while the synthetic graph serves as an integrative aggregator that distills the essential characteristics of the entire graph through parameterized updates, without direct access to the full dataset. This comprehensive framework substantially reduces memory and computational demands, supports parallel computation and resource-efficient collaboration (e.g., through parameter exchange alone for privacy preservation), and readily extends to privacy-preserving scenarios, demonstrating broad applicability in domains such as Internet of Things (IoT) networks, financial technology (fintech) systems, and bioinformatics analyses.

The primary contributions of this paper can be summarized as follows.

- We pioneer the systematic formulation of directed graph distillation as a key paradigm for analyzing ultra-large directed graphs, introducing the Dir-GD framework, which integrates distributed learning principles with directed Graph Neural Networks (GNNs) [29]. This approach enables efficient compression of massive directed graphs while retaining key topological and semantic properties. Through distributed subgraph processing and aggregation, Dir-GD reduces the high memory and computational costs of centralized full-graph training, supporting scalable graph analysis in resource-limited environments.
- We depart from conventional teacher-student distillation by proposing a community-driven strategy that leverages the structural and directional properties of large-scale directed graphs. Using Infomap community detection, our method partitions graphs into cohesive subgraphs that capture modular patterns and directed flows. By applying PageRank-based centrality for node selection and edge-count weighting to preserve asymmetries, Dir-GD delivers high-fidelity compression while maintaining essential semantics. This overcomes biases in prior symmetrization methods, providing a robust solution for direction-sensitive tasks across various domains.
- Through rigorous experiments on benchmark large-scale directed graph datasets, we show that Dir-GD matches full-graph GNN performance at low compression ratios. The framework cuts memory usage and training times, enabling

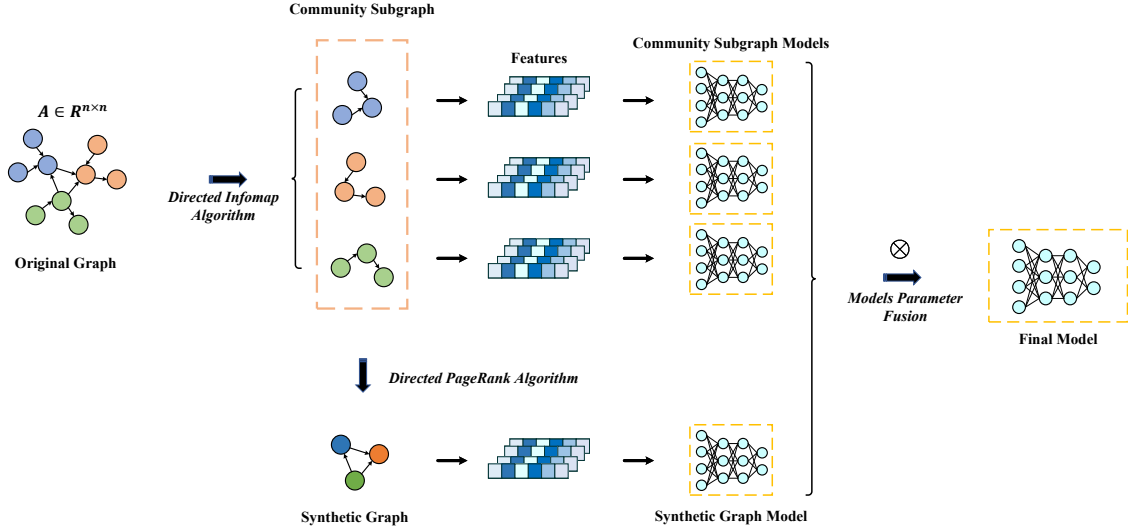


Figure 1: Dir-GD architecture overview, encompassing four primary stages: 1) Community division using directed Infomap to identify modular substructures; 2) Central node selection and synthetic graph building via PageRank to represent community essences; 3) Distributed training on community subgraphs to learn local directed representations; 4) Parameter aggregation for constructing the global model that integrates distributed knowledge.

efficient inference on edge devices and cross-institutional collaboration. These findings underscore Dir-GD’s role in democratizing large-scale graph analytics, with applications in fraud detection, causal inference, and molecular property prediction in distributed environments, and highlight potential avenues for further optimization.

Note that, due to the limited space, partial implementation details, and additional experimental algorithms can be found in the online Appendix.

2 Preliminaries and Problem Statement

This section outlines the foundations of Graph Neural Networks on directed graphs, the principles of graph distillation, and parameter aggregation in distributed learning, providing the basis for understanding the Directed Graph Distillation (Dir-GD) framework. We analyze related work to identify limitations and highlight Dir-GD’s innovations in large-scale directed graph representation learning, including detailed explanations of key concepts and their implications.

2.1 Graph Neural Networks

Graph Neural Networks (GNNs) employ a recursive message-passing framework to aggregate information from nodes and their neighbors, producing embeddings that capture topological dependencies. Formally, consider a directed graph $G = (V, E, X, Y)$, where V is the set of nodes, E is the set of directed edges, $X \in \mathbb{R}^{n \times d}$ is the node feature matrix, and $Y \in \mathbb{R}^{n \times c}$ is the label matrix. The update rule for the l -th layer of a GNN is defined as:

$$\mathbf{h}_v^{(l+1)} = \sigma \left(\mathbf{W}^{(l)} \cdot \text{AGG} \left(\left\{ \mathbf{m}_u^{(l)} \mid u \in \mathcal{N}^-(v) \right\} \cup \left\{ \mathbf{h}_v^{(l)} \right\} \right) \right) \quad (1)$$

where $\mathbf{h}_v^{(l)}$ represents the hidden representation of node v at layer l , $\mathcal{N}^-(v)$ denotes the set of in-degree neighbors of node v , AGG is an aggregation function (e.g., mean, sum, or max), $\mathbf{W}^{(l)}$ is a learnable weight matrix, and σ is a non-linear activation function such as ReLU or sigmoid.

Conventional Graph Neural Network (GNN) [11] architectures, such as Graph Convolutional Networks (GCN) and Graph Attention Networks (GAT) [31], often mitigate edge directionality through symmetric normalization, such as $\bar{A} = \frac{1}{2}(A + A^T)$, where A is the adjacency matrix, introducing representational bias by overlooking directional semantics that are vital for tasks involving flow or causality. To address this limitation, recent approaches [14] separately aggregate in-degree and out-degree neighbors using directed normalization matrices, defined as $\bar{A}^- = D_{\text{in}}^{-1/2} A^T D_{\text{out}}^{-1/2}$ and $\bar{A}^+ = D_{\text{out}}^{-1/2} A D_{\text{in}}^{-1/2}$, where D_{in} and D_{out} are the in-degree and out-degree diagonal matrices, respectively, denoted as “Norm” for brevity. Building on these advancements, some methods incorporate bidirectional attention mechanisms to dynamically weight neighbor contributions based on relevance, allowing for more nuanced representation learning. In this paper, we propose DAGCN-First, a streamlined variant of the DAGCN model [37][30] that restricts aggregation to *first-order* neighbors [36] while effectively capturing bidirectional information from both in-degree and out-degree neighbors, balancing computational efficiency with expressive power. This model introduces a learnable fusion parameter, denoted as α_{fusion} , to adaptively combine representations from forward and reverse propagation paths, enhancing the model’s ability to handle directed asymmetries.

2.2 Graph Distillation

Graph distillation constructs a compact synthetic graph $G_s = (V_s, E_s, X_s, Y_s)$ from a large-scale directed graph $G = (V, E, X, Y)$, enabling a graph neural network (GNN) trained on G_s to closely approximate the performance of one trained on G . This is achieved by optimizing a gradient matching loss:

$$\mathcal{L}_{GM} = \|\nabla_{\theta} \mathcal{L}_{\text{task}}(G, \theta) - \nabla_{\theta} \mathcal{L}_{\text{task}}(G_s, \theta)\|_2 \quad (2)$$

ensuring the synthetic graph's gradient flow aligns with that of the original, allowing the student model to effectively mimic the teacher model. However, these methods focus on undirected graphs, overlooking edge directionality semantics, and often require loading the full graph, which is impractical in distributed settings due to memory, GPU memory, and bandwidth constraints. Our framework extends this by incorporating directionality and distributed processing to handle real-world scalability issues.

2.3 Distributed Learning

Distributed Learning (DL) enables multiple nodes to independently train models on their local data, collaborating to update a global model through the exchange of parameters or gradients [22][4]. This approach facilitates cross-domain knowledge integration while optimizing computational efficiency through parallel processing. The seminal FedAvg algorithm achieves parameter aggregation through iterative communication rounds, as defined by:

$$\theta_g^{(t+1)} = \sum_{k=1}^K w_k \theta_k^{(t+1)} \quad (3)$$

where $\theta_g^{(t+1)}$ represents the global model parameters, $\theta_k^{(t+1)}$ denotes the local updates from node k , and the weights $w_k = n_k/N$ are determined based on the sample size of each node.

The application of DL to graph-structured data further addresses the challenge of computational silos under resource constraints. However, existing methods predominantly target *undirected graphs*, relying on complete graph loading and teacher-student contrastive paradigms. In large-scale real-world deployments, these approaches are often constrained by high communication overheads and struggle to capture the causal and flow semantics embedded in the edge directions of directed graphs. To address these limitations, this work proposes a novel approach that maps directed graph communities to virtual nodes. By leveraging DAGCN-First for local training on each subgraph and introducing a learnable fusion layer to construct a global synthetic graph, our method mitigates the bottlenecks of centralized training. This approach preserves the unique directional semantics of directed graphs, offering a robust solution for directed graph learning in scenarios such as edge intelligence and knowledge graph applications, with additional considerations for non-IID data distributions and convergence guarantees.

2.4 Formulation of Directed Graph Distillation

We formalize directed graph distillation as compressing a large-scale directed graph $G = (V, E, X, Y)$, where V is the node set with $n = |V|$, $E \subseteq V \times V$ is the directed edge set with $m = |E|$, $X \in \mathbb{R}^{n \times d}$ the node features, and $Y \in \mathbb{R}^{n \times c}$ (or partial labels for semi-supervised tasks) the node labels. The objective is to obtain a compact synthetic graph $G_s = (V_s, E_s, X_s, Y_s, W_s)$ with $|V_s| = K \ll n$ and $|E_s| \ll m$,

such that a directed GNN on G_s approximates performance on G , via distributed subgraph processing and parameter aggregation, eschewing centralized full-graph access. For distillation ratio $r \in (0, 1)$, set $K = \lceil rn \rceil$ and minimize the expected task discrepancy:

$$\min_{G_s} \left| \mathbb{E}_{v \in V} [\mathcal{L}(M_g(X_v, \mathcal{N}(v); \theta_g), Y_v)] - \mathbb{E}_{v \in V} [\mathcal{L}(M(X_v, \mathcal{N}(v); \theta), Y_v)] \right| \quad (4)$$

where $\mathcal{L}$ is the task loss (e.g., cross-entropy), $M_g(\cdot; \theta_g)$ the GNN on G_s , $M(\cdot; \theta)$ the reference on G , X_v node v 's features, and $\mathcal{N}(v)$ its directed neighbors. This highlights the distributed paradigm's benefits: scalable distillation of massive directed graphs through subgraph partitioning and local training, directional semantics preservation via community synthesis and weighted aggregation, and efficiency under resource limits without global loading, with potential extensions to handle dynamic graphs or multi-modal features.

3 Methodology

The Dir-GD framework achieves large-scale directed graph distillation through a distributed strategy, comprising five key stages: directed graph Infomap community detection, central node selection and synthetic graph construction, distributed training, parameter fusion with global modeling, and result evaluation. Dir-GD integrates distributed learning with the DAGCN-First model, leveraging community-level distributed training and parameter fusion for efficient knowledge distillation while minimizing communication overhead and computational bottlenecks. Each stage is detailed below, including algorithmic descriptions, complexity analyses, and practical considerations for implementation. The process employs datasets loaded via PyTorch Geometric, with preprocessing for normalization and masking to ensure consistent handling across subgraphs.

3.1 Directed Graph Infomap Community Detection

Community detection partitions the directed graph $G = (V, E, X, Y)$ into K disjoint subgraphs (communities), treated as distributed computing nodes, to capture modular structures and preserve directed edge flows. We use the Infomap algorithm [2], which minimizes the map equation length for community assignment:

$$\min L(M) = q \log q + \sum_{m=1}^M p_m \log p_m + \sum_{m=1}^M \sum_{i \in m} \frac{\tau_i}{1-q} \log \frac{\tau_i}{p_m} \quad (5)$$

where q is the inter-module flow probability, p_m the intra-module flow probability for module m , and τ_i the stationary probability for node i . Based on random walks, it applies to directed graphs and captures directed semantics such as in citation networks, where flows represent knowledge propagation. The target number of communities is $K = \max(1, \int \text{num}_{\text{nodes}} * 0.01)$, adjusted dynamically based on graph size, with distillation ratio r . If the initial partition $K' \neq K$, we iteratively adjust: merge the smallest community to its densest neighbor if $K' > K$; split the largest into two balanced sub-communities if $K' < K$. This ensures balanced sizes for distributed training, promoting load balancing and convergence stability. The complexity is $O(|E| \log |V|)$, supporting distributed computation and providing the basis for subsequent directed graph distillation.

Algorithm 1 outlines the process, with additional boundary node reassignment for optimization.

Algorithm 1 Infomap + Merge-Split Community Adjustment

Require: Directed graph edge index, node count n , target ratio r , max iterations T
Ensure: Community partition C with $|C| = \lceil rn \rceil$

```

1:  $K_{\text{target}} \leftarrow \lceil rn \rceil$ 
2: for  $t = 1$  to  $T$  do
3:   Run Infomap for initial partition  $C', K' \leftarrow |C'|$ 
4:   if  $K' = K_{\text{target}}$  then
5:     return  $C'$ 
6:   end if
7:   while  $K' > K_{\text{target}}$  do
8:     Select smallest community  $C_s$ 
9:     Find neighbors  $\mathcal{N}(C_s)$ 
10:    Compute edge densities  $d(C_s, C_i)$  for  $C_i \in \mathcal{N}(C_s)$ 
11:    Merge  $C_s$  into  $C_t = \arg \max_{C_i} d(C_s, C_i)$ 
12:     $K' \leftarrow K' - 1$ 
13:  end while
14:  while  $K' < K_{\text{target}}$  do
15:    Select largest community  $C_l$ 
16:    Split  $C_l$  via Infomap into  $C_{l1}, C_{l2}$ 
17:    Update  $C' \leftarrow (C' \setminus \{C_l\}) \cup \{C_{l1}, C_{l2}\}$ 
18:     $K' \leftarrow K' + 1$ 
19:  end while
20:  Reassign boundary nodes to optimize metrics
21:  if no improvement then
22:    break
23:  end if
24: end for
25: return  $C'$ 

```

3.2 Central Node Selection and Synthetic Graph Construction

To distill community C_i information, we apply the directed PageRank algorithm [39] to select a central node v_c^i :

$$PR(v) = (1 - d) + d \sum_{u \in \mathcal{N}^-(v)} \frac{PR(u)}{|\mathcal{N}^+(u)|} \quad (6)$$

with damping factor $d = 0.85$, and $\mathcal{N}^-(v)$, $\mathcal{N}^+(v)$ as in/out-neighbors. We solve iteratively (max 100 iterations, tolerance $1e-6$). If the subgraph is not strongly connected, we default to the first node in the community. This approach, prioritizing global influence, outperforms degree centrality by considering the broader network flow.

The synthetic graph $G_s = (V_s, E_s, X_s, Y_s, W_s)$ has V_s as the set of central nodes, $X_s = [X_{v_c^1}; \dots; X_{v_c^K}]$, and $Y_s = [Y_{v_c^1}; \dots; Y_{v_c^K}]$. Edges E_s are weighted: $w_{ij} = e_{ij} / (e_{ij} + e_{ji})$, where e_{ij} counts directed edges between communities, capturing asymmetry and preserving global topological preferences. This design allows for efficient storage using PyTorch Geometric’s SparseTensor, facilitating scalability. The process runs in parallel with complexity $O(K \cdot (|E|/K + |V|/K \log |V|/K))$, outperforming full-graph $O(|E| + |V| \log |V|)$, ensuring the synthetic graph approximates the original topology and boosts downstream GNN performance. Algorithm 2 outlines the process, with considerations for handling sparse communities.

3.3 Distributed Training

On each community subgraph, we extract node features, edges, and labels to form independent data objects, and train locally using the DAGCN-First model. This model captures directed graph asymmetries via directed normalized adjacency matrices and learnable mixing parameters, combining forward (source-to-target) and

Algorithm 2 Central Node Selection + Synthetic Graph Construction

Require: Community list $C = \{C_1, \dots, C_K\}$, edge index, node features X , node labels Y
Ensure: Synthetic graph $G_s = (X_s, Y_s, E_s, W_s)$

```

1: Initialize  $V_c \leftarrow []$ 
2: for each community  $C_i \in C$  in parallel do
3:   Extract subgraph  $G_i$ 
4:   Compute PageRank  $PR$  on  $G_i$ 
5:   Select  $v_c^i = \arg \max_{v \in C_i} PR(v)$  or fallback to  $C_i[0]$ 
6:   Append  $v_c^i$  to  $V_c$ 
7: end for
8:  $X_s \leftarrow X[V_c]$ ,  $Y_s \leftarrow Y[V_c]$ 
9: Initialize edge count dict  $e \leftarrow \emptyset$ 
10: for each edge  $(u, v) \in \text{edge index do}$ 
11:   Find communities  $i$  for  $u, j$  for  $v$ 
12:   if  $i \neq j$  then
13:      $e_{(i,j)} \leftarrow e_{(i,j)} + 1$ 
14:   end if
15: end for
16: Initialize  $E_s \leftarrow []$ ,  $W_s \leftarrow []$ 
17: for each non-zero  $(i, j)$ ,  $i \neq j$  do
18:   Compute total edges  $total = e_{(i,j)} + e_{(j,i)}$ 
19:   if  $total > 0$  then
20:     Add  $(i, j)$  to  $E_s$ , weight  $e_{(i,j)} / total$  to  $W_s$ 
21:     if  $e_{(j,i)} > 0$  then
22:       Add  $(j, i)$  to  $E_s$ , weight  $e_{(j,i)} / total$  to  $W_s$ 
23:     end if
24:   end if
25: end for
26: Store  $E_s, W_s$  as SparseTensor
27: return  $G_s$ 

```

reverse (target-to-source) propagation paths. Inputs include subgraph feature matrix $X_i \in \mathbb{R}^{|C_i| \times d}$ (with feature dimension d), edge index $E_i \in \mathbb{N}^{2 \times |E(C_i)|}$, and labels $Y_i \in \mathbb{N}^{|C_i| \times 1}$. The model comprises an input layer, 1-2 hidden layers, and an output layer, each using DAGCNConv operations, with hyperparameters tuned for optimal performance. Training employs PyTorch Lightning for efficient management, incorporating early stopping based on training loss and ReduceLROnPlateau (a PyTorch scheduler that reduces the learning rate when a monitored metric plateaus) scheduling to adapt learning rates.

The DAGCNConv layer is formalized as follows: given input $H^{(l)} \in \mathbb{R}^{m \times h}$, where $m = |C_i|$ is the number of nodes in community C_i and h is the hidden dimension, construct sparse adjacency tensor adj from edge set E_i , optionally weighted. Compute directed normalization $\text{adj}_{\text{norm}} = D_{\text{out}}^{-1/2} \text{adj} D_{\text{in}}^{-1/2}$, where D_{out} and D_{in} are diagonal matrices of out-degrees and in-degrees, respectively. For nodes with zero in- or out-degree, an infinity mask is applied to set corresponding entries in $D_{\text{out}}^{-1/2}$ and $D_{\text{in}}^{-1/2}$ to infinity during computation; subsequent masked multiplication effectively zeros these normalization factors, preventing erroneous propagation while preserving sparsity. Similarly, compute reverse adjacency $\text{adj}_t = \text{adj}^T$ and normalize as $\text{adj}_{t,\text{norm}} = D_{\text{in}}^{-1/2} \text{adj}_t D_{\text{out}}^{-1/2}$. Layer updates are computed as $H_{\text{src-to-dst}}^{(l+1)} = W_{\text{src-to-dst}} (\text{adj}_{\text{norm}} H^{(l)})$ and $H_{\text{dst-to-src}}^{(l+1)} = W_{\text{dst-to-src}} (\text{adj}_{t,\text{norm}} H^{(l)})$, where $W_{\text{src-to-dst}}, W_{\text{dst-to-src}} \in \mathbb{R}^{h' \times h}$ and h' is the output dimension. Final aggregation is:

$$H^{(l+1)} = \sigma(\alpha) \cdot H_{\text{src-to-dst}}^{(l+1)} + (1 - \sigma(\alpha)) \cdot H_{\text{dst-to-src}}^{(l+1)} \quad (7)$$

where α is learnable, σ is sigmoid (hereafter $\sigma(\alpha)$ denoted as α). This applies per layer, with multi-layer stacking for deeper representations. Jumping Knowledge fuses outputs: $H = \max(H^{(1)}, \dots, H^{(L)})$ (element-wise max). L2 normalization and dropout enhance generalization and prevent overfitting. The distributed design enables parallel execution on multiple nodes, with complexity $O(K \cdot (|E|/K) \cdot L)$,

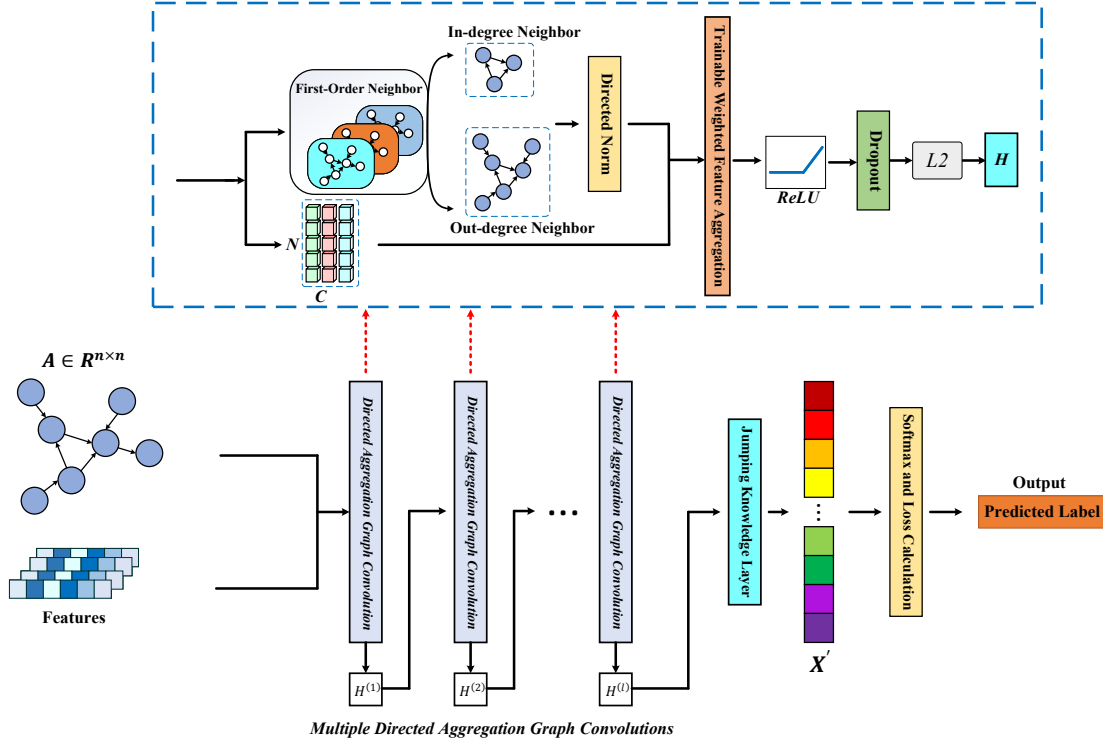


Figure 2: The DAGCN-First architecture comprises Directed Aggregation Graph Convolutional Networks (DAGCNs). It takes adjacency matrix $A \in \mathbb{R}^{n \times n}$ and feature matrix $X \in \mathbb{R}^{N \times C}$. Each DAGCN layer aggregates first-order neighbor features, applies directional normalization and weighted aggregation, followed by ReLU activation, dropout, and L2 normalization, producing representations $H^{(1)}, H^{(2)}, \dots, H^{(l)}$. Layer outputs are fused via Jumping Knowledge, then passed to a softmax classifier for node predictions, enabling effective capture of directed semantics.

lower than full-graph $O(|E| \cdot L)$, and supports early stopping for efficiency. Algorithm 4 outlines the process (see Appendix Algorithm 4).

3.4 Parameter Fusion and Global Model

The global model M_g is pre-trained on synthetic graph G_s for initialization in Dir-GD. For each local model M_i , we introduce learnable fusion weight β (with sigmoid ensuring $\sigma(\beta) \in [0, 1]$), and perform per-parameter weighted averaging:

$$\theta_g \leftarrow (1 - \beta) \theta_g + \beta \theta_i \quad (8)$$

(Hereafter $\sigma(\beta)$ denoted as β). This adapts to community heterogeneity, integrating distributed knowledge with efficient communication (parameter exchange only). Post-fusion, global evaluation uses batch mode, partitioning the graph to compute predictions without full loading. Total complexity is $O(|E| + K \cdot (n/K) \cdot L)$, superior to centralized methods. Layer-wise averaging with learnable β enhances generalization, robust to heterogeneous updates via gradient analysis. Compared to distributed methods, Dir-GD accounts for directed graph semantics, avoiding performance drops from simple averaging. In the Dir-GD-NoSynth variant (see Experiments' Ablation Study section), random initialization replaces pre-training,

reducing overhead while maintaining efficacy. Algorithm 3 provides the pseudocode, with fine-tuning steps for refinement.

Algorithm 3 Parameter Fusion

Require: Global parameters θ_g , local parameters $\{\theta_i\}_{i=1}^K$, fusion parameter β
Ensure: Updated global parameters θ_g
1: **for** each $i = 1$ to K **do**
2: **for** each key in θ_g **do**
3: $\theta_g[\text{key}] \leftarrow (1 - \beta) \cdot \theta_g[\text{key}] + \beta \cdot \theta_i[\text{key}]$
4: **end for**
5: **end for**
6: Fine-tune θ_g on synthetic graph
7: **return** θ_g

3.5 Result Testing

This stage evaluates global model M_g on directed graph $G = (V, E, X, Y)$ test set for node classification via accuracy. Test set is marked by mask $\text{test_mask} \in \{0, 1\}^{|V|}$, with accuracy:

$$\text{acc} = \frac{\sum_b \sum_{i \in V_b} \text{test_mask}_b[i] \cdot \mathbb{I}(Y_b^{\text{pred}}[i] = Y_b[i])}{N_{\text{test}}} \quad (9)$$

where $V_b \subseteq V$ is batch nodes, $Y_b^{\text{pred}} = \arg \max (M_g(X_b, E'_b))$, $N_{\text{test}} = \sum \text{test_mask}[i]$, $\mathbb{I}(\cdot)$ indicator. Batch strategy dynamically builds

subgraphs, extracting test features $X_b \in \mathbb{R}^{|V_b| \times d}$ and edges E'_b , reducing memory and enabling parallel inference. Complexity is $O(\sum_b (|E_b|Lh + |V_b|h^2 + |E_b|))$, lower than full-graph $O(|E|Lh + |V|h^2)$ for $B \ll |V|$, with $|E_b| \leq |E| \cdot (B/|V|)^2$, L layers, h hidden dim. Algorithm 5 outlines the process (see Appendix Algorithm 5), with options for metric extensions like F1-score.

4 Experiments

This study evaluates semi-supervised node classification, where given graph $G = (V, E)$ with adjacency A , features $X \in \mathbb{R}^{N \times C}$ ($N = |V|$, C dimensions), and labeled subset $V_L \subset V$ where $|V_L| \ll |V|$, the goal is to predict labels for unlabeled $V_U = V \setminus V_L$ using V_L , X , and A . We employ the DAGCN-First model, which aggregates first-order in- and out-neighbor information to enhance node representations, capturing local and global structures. By modeling directionality and multi-hop relations, DAGCN-First leverages graph structure to improve classification. Dir-GD implementation details are available at: <https://github.com/CYang798/Dir-GD-Directed-Graph-Distillation>.

Table 1: Dataset Details

Dataset	#Nodes	#Edges	#Features	#Classes	%Unidirectional Edges	Graph Type
DIRECTED_ROMAN_EMPIRE	22662	44363	300	18	65.24	Homogeneous
DIRECTED_QUESTIONS	48921	156960	301	2	98.38	Heterogeneous
OGBN-ARXIV	169343	1166243	128	40	99.27	Homogeneous
GENIUS	421961	984979	12	2	93.27	Homogeneous
SOC-POKEC-RELATIONSHIPS	1632803	30622564	59	5	62.69	Homogeneous
DGRAPHFIN	3700550	4300999	17	2	92.40	Heterogeneous

4.1 Experimental Setup

4.1.1 Datasets. Dataset details are in Table 1, covering scales from tens of thousands to millions of nodes, with varying directed edge ratios, selected to represent diverse real-world scenarios including historical networks, question-answering graphs, academic citations, social profiles, and financial transactions. Datasets are preprocessed using custom loaders to handle directed heterophilous graphs and ensure uniform train/val/test splits.

4.1.2 Hyperparameter Tuning. We grid-searched weights α and β on the validation set over $[0, 1]$ with step 0.05, fixing LR at 0.01, JK as MAX, and Norm=True. For fair comparison, all baselines' hyperparameters were similarly optimized via grid search on recommended ranges, tuned on validation sets, with multiple runs for stability. The search includes hidden dimensions and number of layers, balancing model capacity with efficiency.

4.1.3 Loss Function. The cross-entropy loss measures prediction-ground truth discrepancy for parameter optimization:

$$L = - \sum_{i=1}^N y_i \log(p(y_i|x_i)), \quad (10)$$

where y_i is the true label, $p(y_i|x_i)$ the predicted probability for sample x_i . Model outputs z yield probabilities via softmax, optimizing classification on training data, with regularization terms for robustness.

4.2 Experimental Results

Since we are the first to implement directed graph distillation, we no longer conduct comparative experiments with other undirected graph methods in the experimental section. Results are in Table 2, showing Dir-GD achieves high accuracy at low distillation ratios across datasets. We categorize into homogeneous [34](DIRECTED-ROMAN-EMPIRE, GENIUS, OGBN-ARXIV, SOC-POKEC-RELATIONSHIPS) and heterogeneous (DIRECTED-QUESTIONS, DGRAPHFIN) graphs. For homogeneous: DIRECTED-ROMAN-EMPIRE at 0.001 yields $86.57\% \pm 1.51\%$, demonstrating resilience in multi-class settings; GENIUS at 0.001 yields $79.80\% \pm 0.98\%$, highlighting binary classification stability; OGBN-ARXIV [13] at 0.001 yields $67.51\% \pm 1.80\%$, reflecting challenges in dense citation networks; SOC-POKEC-RELATIONSHIPS at 0.0005 yields $96.24\% \pm 1.13\%$, showcasing scalability in social graphs. For heterogeneous: DIRECTED-QUESTIONS at 0.001 yields $97.11\% \pm 0.78\%$, indicating strong performance in query-response structures; DGRAPHFIN at 0.0001 yields $66.88\% \pm 0.72\%$, underscoring utility in imbalanced financial data. Dir-GD preserves topological consistency in homogeneous graphs and directional semantics in heterogeneous ones, approaching full-graph performance at high compression while reducing costs, with trends suggesting diminishing returns at extreme ratios.

Table 2: Dir-GD Results Under Optimal Hyperparameters

Dataset	DIRECTED_ROMAN_EMPIRE			DIRECTED_QUESTIONS		
Params	Ratio	Hyperparameters	Acc	Ratio	Hyperparameters	Acc
Dir-GD	0.01	$\alpha:0.25$ $\beta:0.25$ hidden:128 layers:3	81.26 $\pm$ 2.87	0.01	$\alpha:0.5$ $\beta:0.5$ hidden:128 layers:4	97.06 $\pm$ 1.17
	0.001	$\alpha:0.5$ $\beta:0.5$ hidden:128 layers:3	86.57 $\pm$ 1.51	0.001	$\alpha:0.25$ $\beta:0.25$ hidden:128 layers:2	97.11 $\pm$ 0.78
	0.005	$\alpha:0.25$ $\beta:0.5$ hidden:128 layers:3	81.17 $\pm$ 2.49	0.005	$\alpha:0.25$ $\beta:0.25$ hidden:128 layers:2	97.02 $\pm$ 1.32
Dataset	GENIUS			OGBN-ARXIV		
Params	Ratio	Hyperparameters	Acc	Ratio	Hyperparameters	Acc
Dir-GD	0.001	$\alpha:0.5$ $\beta:0.5$ hidden:128 layers:2	79.80 $\pm$ 0.98	0.001	$\alpha:0.25$ $\beta:0.75$ hidden:256 layers:3	67.51 $\pm$ 1.80
	0.0001	$\alpha:0.25$ $\beta:0.5$ hidden:128 layers:3	79.80 $\pm$ 0.28	0.0001	$\alpha:0.25$ $\beta:0.25$ hidden:128 layers:4	66.81 $\pm$ 2.01
	0.0005	$\alpha:0.25$ $\beta:0.5$ hidden:128 layers:3	79.80 $\pm$ 0.22	0.0005	$\alpha:0.75$ $\beta:0.5$ hidden:128 layers:4	66.40 $\pm$ 0.96
Dataset	DGRAPHFIN			SOC-POKEC-RELATIONSHIPS		
Params	Ratio	Hyperparameters	Acc	Ratio	Hyperparameters	Acc
Dir-GD	0.0001	$\alpha:0.5$ $\beta:0.5$ hidden:128 layers:2	66.88 $\pm$ 0.72	0.0001	$\alpha:0.25$ $\beta:0.25$ hidden:128 layers:2	91.13 $\pm$ 0.90
	0.0005	$\alpha:0.25$ $\beta:0.25$ hidden:128 layers:4	65.82 $\pm$ 1.21	0.0005	$\alpha:0.25$ $\beta:0.25$ hidden:128 layers:2	96.24 $\pm$ 1.13

4.3 Ablation Study

To assess component contributions in Dir-GD, we conduct ablation experiments evaluating impacts on global model M_g performance.

We introduce the Dir-GD-NoSynth variant, which omits synthetic graph pre-training, initializing the global model randomly before fusing local updates. In full Dir-GD, the synthetic graph is built via community detection and central node selection, pre-trained to capture inter-community relations, initializing θ_g . Global and community models share configurations (input features, hidden dims, layers, output classes) for parameter matching. Fusion uses weighted averaging:

$$\theta_g \leftarrow (1 - \beta) \theta_g + \beta \theta_i \quad (11)$$

with initial θ_g contribution decaying to $(1 - \beta)^K \theta_g$ over K iterations. Since $\beta \in (0, 1)$, $(1 - \beta)^K$ approaches zero rapidly with large K , indicating pre-training sets initial direction, diminishing with iterations, but providing a structured starting point for convergence.

The ablation variant, Dir-GD-NoSynth, removes synthetic graph construction and pre-training, starting with random θ_g , updating via community training and fusion, keeping other settings identical. Results in Tables 2 and 3 show Dir-GD slightly outperforms Dir-GD-NoSynth on most datasets, e.g., OGBN-ARXIV at 0.001: 67.51 ± 1.80 vs. 66.42 ± 1.73 ; DIRECTED_QUESTIONS at 0.001: 97.11 ± 0.78 vs. 97.09 ± 0.08 ; GENIUS at 0.001: 79.80 ± 0.98 vs. 79.79 ± 0.20 . However, Dir-GD-NoSynth excels on DIRECTED_ROMAN_EMPIRE (0.001: 88.60 ± 2.71 vs. 86.57 ± 1.51) and DGRAPHFIN (0.0005: 67.00 ± 1.27 vs. 65.82 ± 1.21). Overall, comprehensively considering accuracy, GPU memory, memory storage, and training efficiency, we recommend Dir-GD-NoSynth for its efficiency and comparable accuracy, avoiding biases from synthetic approximations, especially when not excessively pursuing precision.

4.4 Discussion

Results indicate Dir-GD and Dir-GD-NoSynth achieve high accuracy at low distillation ratios, compressing large directed graphs while maintaining performance. Dir-GD time complexity is $O(|E| \log |V| + K \cdot (|E|/K) \cdot L)$, space $O(|V| + |E|/K)$ (K communities, $|E|$ edges, $|V|$ nodes, L GNN layers). Dir-GD-NoSynth reduces time to $O(|E| \log |V| + K \cdot (|E|/K) \cdot L - |V_s| \cdot L)$, space to $O(|V| + |E|/K - |V_s|)$ ($|V_s|$ synthetic nodes), improving efficiency. Dir-GD-NoSynth outperforms Dir-GD on some datasets due to synthetic graph properties in Dir-GD. The synthetic graph approximates inter-community directed flows using carefully weighted bidirectional edges, effectively guiding initialization. When closely aligned with the original topology, it significantly aids convergence; however, mismatches, such as in highly asymmetric graphs, may impede effective fusion and reduce accuracy. This highlights the pronounced sensitivity of synthetic graphs in directed settings, with Dir-GD-NoSynth demonstrating superior robustness through distributed aggregation alone, proving particularly effective in diverse heterogeneous environments.

5 Conclusion

This work introduces the Dir-GD framework and its variant Dir-GD-NoSynth, pioneering directed graph distillation as a scalable method for compressing large directed graphs. By integrating distributed learning, we enable efficient distillation of million-scale directed graphs, bypassing traditional teacher-student paradigms

Table 3: Dir-GD-NoSynth Results Under Optimal Hyperparameters

Dataset	DIRECTED_ROMAN_EMPIRE			DIRECTED_QUESTIONS		
	Ratio	Hyperparameters	Acc (%)	Ratio	Hyperparameters	Acc (%)
Dir-GD	0.01	$\alpha: 0.25$ $\beta: 0.25$ Hidden: 128 Layers: 4	80.71 ± 3.21	0.01	$\alpha: 0.25$ $\beta: 0.5$ Hidden: 128 Layers: 4	97.03 ± 1.12
	0.001	$\alpha: 0.5$ $\beta: 0.25$ Hidden: 128 Layers: 1	88.60 ± 2.71	0.001	$\alpha: 0.25$ $\beta: 0.75$ Hidden: 128 Layers: 2	97.09 ± 0.08
	0.005	$\alpha: 0.25$ $\beta: 0.25$ Hidden: 128 Layers: 4	85.23 ± 2.83	0.005	$\alpha: 0.25$ $\beta: 0.5$ Hidden: 128 Layers: 2	97.00 ± 1.02
Dataset	GENIUS			OGBN-ARXIV		
	Ratio	Hyperparameters	Acc (%)	Ratio	Hyperparameters	Acc (%)
Dir-GD	0.001	$\alpha: 0.25$ $\beta: 0.5$ Hidden: 128 Layers: 2	79.79 ± 0.20	0.001	$\alpha: 0.25$ $\beta: 0.25$ Hidden: 256 Layers: 3	66.42 ± 1.73
	0.0001	$\alpha: 0.25$ $\beta: 0.25$ Hidden: 128 Layers: 2	79.79 ± 0.11	0.0001	$\alpha: 0.5$ $\beta: 0.25$ Hidden: 256 Layers: 3	66.29 ± 1.77
	0.0005	$\alpha: 0.25$ $\beta: 0.5$ Hidden: 128 Layers: 2	79.79 ± 0.23	0.0005	$\alpha: 0.75$ $\beta: 0.75$ Hidden: 256 Layers: 5	66.96 ± 0.14
Dataset	DGRAPHFIN			SOC-POKEC-RELATIONSHIPS		
	Ratio	Hyperparameters	Acc (%)	Ratio	Hyperparameters	Acc (%)
Dir-GD	0.0001	$\alpha: 0.5$ $\beta: 0.25$ Hidden: 128 Layers: 2	66.92 ± 0.78	0.0001	$\alpha: 0.25$ $\beta: 0.25$ Hidden: 128 Layers: 2	90.05 ± 0.72
	0.0005	$\alpha: 0.25$ $\beta: 0.25$ Hidden: 128 Layers: 3	67.00 ± 1.27	0.0005	$\alpha: 0.25$ $\beta: 0.25$ Hidden: 128 Layers: 2	98.22 ± 1.49

through community-driven training and parameter fusion. This preserves directional semantics, ensures scalability and efficiency, and is particularly suited for edge computing and cross-institution scenarios where data privacy and resource constraints are paramount. Decomposing large graphs into subgraphs and aggregating local knowledge, Dir-GD reduces overhead without full loading, e.g., on SOC-POKEC-RELATIONSHIPS at 0.001 ratio, accuracy exceeds 91%, with NoSynth variant often providing slight improvements due to avoided initialization biases. Limitations include adaptability to heterogeneity like sparsity or irregular communities, potential sensitivity to hyperparameter choices, and assumptions of static graphs. Extending to temporal dynamics [27][10], adaptive partitioning, or hybrid distributed setups could broaden applications in social analysis, fraud detection, protein prediction, recommendations, and traffic optimization, including social recommendations via user-friend graphs for mobile GNN deployments, real-time congestion analysis in road network traffic prediction, and weather forecasting in meteorological networks to reduce simulation times from hours to minutes, paving the way for future research in scalable graph learning.

Acknowledgement

This work was funded in part by the National Natural Science Foundation of China (Grant No. 62477029, 61702317), the Ministry of Education Humanities and Social Sciences Research Youth Fund Project (Grant No. 22YJCZH046), and the Fundamental Research Funds for the Central Universities, China (Grant No. GK202505028).

References

- [1] Muhammad Asad, Ahmed Moustafa, and Takayuki Ito. 2020. Fedopt: Towards communication efficiency and privacy preservation in federated learning. *Applied Sciences* 10, 8 (2020), 2864.
- [2] Esteban Vargas Bernal, Mason A Porter, and Joseph H Tien. 2024. Adapting InfoMap to Absorbing Random Walks Using Absorption-Scaled Graphs\ast. *SIAM JOURNAL ON APPLIED DYNAMICAL SYSTEMS* 23, 3 (2024), 2557–2592.
- [3] Fahao Chen, Peng Li, Toshiaki Miyazaki, and Celimuge Wu. 2021. Fedgraph: Federated graph learning with intelligent sampling. *IEEE Transactions on Parallel and Distributed Systems* 33, 8 (2021), 1775–1786.
- [4] Ming Chen, Zhewei Wei, Bolin Ding, Yaliang Li, Ye Yuan, Xiaoyong Du, and Ji-Rong Wen. 2020. Scalable graph neural networks via bidirectional propagation. *Advances in neural information processing systems* 33 (2020), 14556–14566.
- [5] Wei-Lin Chiang, Xuanqing Liu, Si Si, Yang Li, Samy Bengio, and Cho-Jui Hsieh. 2019. Cluster-gcn: An efficient algorithm for training deep and large graph convolutional networks. In *Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining*. 257–266.
- [6] Michaël Defferrard, Xavier Bresson, and Pierre Vandergheynst. 2016. Convolutional neural networks on graphs with fast localized spectral filtering. *Advances in neural information processing systems* 29 (2016).
- [7] Chen Gao, Xiang Wang, Xiangnan He, and Yong Li. 2022. Graph neural networks for recommender system. In *Proceedings of the fifteenth ACM international conference on web search and data mining*. 1623–1625.
- [8] Xinyi Gao, Tong Chen, Yilong Zang, Wentao Zhang, Quoc Viet Hung Nguyen, Kai Zheng, and Hongzhi Yin. 2024. Graph condensation for inductive node representation learning. In *2024 IEEE 40th International Conference on Data Engineering (ICDE)*. IEEE, 3056–3069.
- [9] David F Gleich. 2015. PageRank beyond the web. *siam REVIEW* 57, 3 (2015), 321–363.
- [10] Alessio Gravina and Davide Bacciu. 2024. Deep Learning for Dynamic Graphs: Models and Benchmarks. *IEEE Trans. Neural Networks Learn. Syst.* 35, 9 (September 2024), 11788–11801. <https://doi.org/10.1109/TNNLS.2024.3379735>
- [11] Will Hamilton, Zhitaoying, and Jure Leskovec. 2017. Inductive representation learning on large graphs. *Advances in neural information processing systems* 30 (2017).
- [12] Chaoyang He, Keshav Balasubramanian, Emir Ceyani, Carl Yang, Han Xie, Lichao Sun, Lifang He, Liangwei Yang, S Yu Philip, Yu Rong, et al. 2021. Fedgraphnn: A federated learning benchmark system for graph neural networks. In *ICLR 2021 Workshop on Distributed and Private Machine Learning (DPML)*.
- [13] Weihua Hu, Matthias Fey, Marinka Zitnik, Yuxiao Dong, Hongyu Ren, Bowen Liu, Michele Catasta, and Jure Leskovec. 2020. Open graph benchmark: Datasets for machine learning on graphs. *Advances in neural information processing systems* 33 (2020), 22118–22133.
- [14] Jincheng Huang and Xiaofeng Zhu. 2025. Adaptive node-level weighted learning for directed graph neural network. *Neural Networks* 187 (2025), 107393.
- [15] Wei Jin, Xianfeng Tang, Haoming Jiang, Zheng Li, Danqing Zhang, Jiliang Tang, and Bing Yin. 2022. Condensing graphs via one-step gradient matching. In *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 720–730.
- [16] Wei Jin, Lingxiao Zhao, Shichang Zhang, Yozen Liu, Jiliang Tang, and Neil Shah. 2021. Graph Condensation for Graph Neural Networks. *CoRR* abs/2110.07580 (2021). <https://arxiv.org/abs/2110.07580>
- [17] Wei Jin, Lingxiao Zhao, Shichang Zhang, Yozen Liu, Jiliang Tang, and Neil Shah. 2022. Graph Condensation for Graph Neural Networks. In *International Conference on Learning Representations*. <https://openreview.net/forum?id=WLEx3Jo4QaB>
- [18] Thomas N. Kipf and Max Welling. 2017. Semi-Supervised Classification with Graph Convolutional Networks. In *International Conference on Learning Representations*. <https://openreview.net/forum?id=SJU4ayYgl>
- [19] Guillaume Lachaud, Patricia Conde-Cespedes, and Maria Trocan. 2022. Graph neural networks-based multilabel classification of citation network. In *Asian Conference on Intelligent Information and Database Systems*. Springer, 128–140.
- [20] Xinglin Li, Kun Wang, Hanhui Deng, Yuxuan Liang, and Di Wu. 2023. Attend Who is Weak: Enhancing Graph Condensation via Cross-Free Adversarial Training. *CoRR* abs/2311.15772 (2023). <https://doi.org/10.48550/arXiv.2311.15772>
- [21] Mengyang Liu, Shanchuan Li, Xinshi Chen, and Le Song. 2022. Graph condensation via receptive field distribution matching. *arXiv preprint arXiv:2206.13697* (2022).
- [22] Rui Liu, Pengwei Xing, Zichao Deng, Anran Li, Cuntai Guan, and Han Yu. 2024. Federated graph neural networks: Overview, techniques, and challenges. *IEEE transactions on neural networks and learning systems* (2024).
- [23] Yang Liu, Deyu Bo, and Chuan Shi. 2024. Graph Distillation with Eigenbasis Matching. In *ICML*. <https://openreview.net/forum?id=DYN66JJC19>
- [24] Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson, and Blaise Agüera y Arcas. 2017. Communication-efficient learning of deep networks from decentralized data. In *Artificial intelligence and statistics*. PMLR, 1273–1282.
- [25] Vasimuddin Md, Sanchit Misra, Guixiang Ma, Ramanarayan Mohanty, Evangelos Georganas, Alexander Heinecke, Dhiraj Kalamkar, Nesreen K Ahmed, and Sasikanth Avancha. 2021. Distgm: Scalable distributed training for large-scale graph neural networks. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. 1–14.
- [26] Nicolas Michel, Maorong Wang, Ling Xiao, and Toshihiko Yamasaki. 2023. Rethinking Momentum Knowledge Distillation in Online Continual Learning. *CoRR* abs/2309.02870 (2023). <https://doi.org/10.48550/arXiv.2309.02870>
- [27] Emanuele Rossi, Ben Chamberlain, Fabrizio Frasca, Davide Eynard, Federico Monti, and Michael M. Bronstein. 2020. Temporal Graph Networks for Deep Learning on Dynamic Graphs. *CoRR* abs/2006.10637 (2020). <https://arxiv.org/abs/2006.10637>
- [28] Miru Tang, Baiqing Li, and Hongming Chen. 2023. Application of message passing neural networks for molecular property prediction. *Current Opinion in Structural Biology* 81 (2023), 102616.
- [29] Veronika Thost and Jie Chen. 2021. Directed Acyclic Graph Neural Networks. In *International Conference on Learning Representations*. <https://openreview.net/forum?id=JbuYF437WB6>
- [30] Zekun Tong, Yuxuan Liang, Changsheng Sun, David S. Rosenblum, and Andrew Lim. 2020. Directed Graph Convolutional Network. *CoRR* abs/2004.13970 (2020). <https://arxiv.org/abs/2004.13970>
- [31] Petar Velickovic, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Lio, Yoshua Bengio, et al. 2017. Graph attention networks. *stat* 1050, 20 (2017), 10–48550.
- [32] Petar Velicković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Lio, and Yoshua Bengio. 2018. Graph Attention Networks. In *International Conference on Learning Representations*. <https://openreview.net/forum?id=rJXmpikCZ>
- [33] Cheng Wan, Youjie Li, Cameron R. Wolfe, Anastasios Kyrillidis, Nam Sung Kim, and Yingyan Lin. 2022. PipeGCN: Efficient Full-Graph Training of Graph Convolutional Networks with Pipelined Feature Communication. In *International Conference on Learning Representations*. <https://openreview.net/forum?id=kSwqMH0zn1F>
- [34] Xiao Wang, Houye Ji, Chuan Shi, Bai Wang, Yanfang Ye, Peng Cui, and Philip S Yu. 2019. Heterogeneous graph attention network. In *The world wide web conference*. 2022–2032.
- [35] Lirong Wu, Haitao Lin, Yufei Huang, and Stan Z Li. 2022. Knowledge distillation improves graph structure augmentation for graph neural networks. *Advances in Neural Information Processing Systems* 35 (2022), 11815–11827.
- [36] Zonghan Wu, Shirui Pan, Fengwen Chen, Guodong Long, Chengqi Zhang, and S Yu Philip. 2020. A comprehensive survey on graph neural networks. *IEEE transactions on neural networks and learning systems* 32, 1 (2020), 4–24.
- [37] Ce Yang, Fei Hao, Wangyang Yu, Carmen De Maio, and Liang Zhao. 2025. DAGCN: A Novel Directed Aggregation Graph Convolutional Network for Optimizing Receptive Field Expansion. *IEEE Transactions on Network Science and Engineering* (2025). doi:10.1109/TNSE.2025.3605805
- [38] Hanqing Zeng, Hongkuan Zhou, Ajitesh Srivastava, Rajgopal Kannan, and Viktor K. Prasanna. 2019. GraphSAINT: Graph Sampling Based Inductive Learning Method. *CoRR* abs/1907.04931 (2019). <http://arxiv.org/abs/1907.04931>
- [39] Panpan Zhang, Tiandong Wang, and Jun Yan. 2022. PageRank centrality and algorithms for weighted, directed networks. *Physica A: Statistical Mechanics and its Applications* 586 (2022), 126438.
- [40] Jie Zhou, Ganqu Cui, Shengding Hu, Zhengyan Zhang, Cheng Yang, Zhiyuan Liu, Lifeng Wang, Changcheng Li, and Maosong Sun. 2020. Graph neural networks: A review of methods and applications. *AI open* 1 (2020), 57–81.

Appendix

A Related Work

To situate the proposed Directed Graph Distillation (Dir-GD) framework within the broader context of graph distillation and distributed learning methodologies, we conduct a comparative analysis with established baselines, evaluating aspects such as graph type support, resource demands, compression efficiency, and underlying mechanisms, as presented in Table 4. Prevailing distillation approaches primarily address undirected graphs and depend on centralized access to the entire graph, frequently employing teacher-student frameworks that entail substantial computational costs and overlook directional semantics. In contrast, Dir-GD innovates by leveraging distributed subgraph partitioning and parameter aggregation to facilitate scalable, resource-efficient distillation of directed graphs without requiring global data access. A concise overview of the key methodologies follows, expanded with discussions on their

Table 4: Dir-GD vs. Related Methods

Method	Graph Type	Resource Needs	Max Reduction (%)	Max Nodes (M)	Core Mechanism
GCond	Undirected	Full Graph	99.9	0.716	Gradient Matching
GDEM	Undirected	Full Graph	99.5	0.233	Spectral Alignment
DosCond	Undirected	Full Graph	90	0.233	Single-Step Matching
KDGA	Undirected	Full Graph	N/A	0.034	KD Augmentation
MCond	Undirected	Original Inference	98.21	0.716	Node Mapping
MKD OCL	Non-Graph	Full Graph	N/A	N/A	Dynamic KD
Shock Absorber	Undirected	Full Graph	N/A	0.169	Adversarial Training
FedGraphNN	Undirected	Distributed Clients	N/A	0.716	Federated Benchmark
DistGNN	Undirected	Distributed	N/A	1.0	Partitioned Training
PipeGCN	Undirected	Distributed	N/A	0.5	Pipelined Aggregation
Dir-GD (Ours)	Directed	Distributed Subgraphs	99.9	3.7	Distributed Fusion

Note: Max Reduction (%) indicates the highest reported percentage reduction in graph size (nodes/edges) while maintaining comparable GNN performance (e.g., >90% accuracy retention). Max Nodes (M) denotes the size of the largest dataset processed (in millions of nodes).

strengths, limitations, and relevance to our work: GCond [16][21] employs bilevel optimization for gradient alignment in undirected graphs, ensuring consistent training dynamics but requiring full-graph computation, which limits scalability in directed large-scale settings. GDEM [23] focuses on spectral alignment to preserve eigenstructure and features, minimizing deviations in undirected settings at the cost of centralized processing, lacking support for directionality that Dir-GD addresses. DosCond [15] introduces single-step gradient matching with probabilistic edge modeling for efficient undirected graph condensation, achieving up to 90% compression but without considering directed flows. KDGA [35] integrates knowledge distillation with graph augmentation to enhance transfer in undirected graphs, though it demands complete data access and does not extend to distributed environments. MCond [8] targets inductive settings via one-to-many node mappings, yielding 98.21% compression for undirected graphs during inference, but overlooks distributed training needs. MKD-OCL [26] applies momentum-based updates to mitigate forgetting in online knowledge distillation, primarily for non-graph domains, with potential adaptations for graphs but no directionality focus. Shock Absorber [20] bolsters stability through cross-modal adversarial training in undirected graphs, but still necessitates full-graph loading, contrasting Dir-GD’s distributed approach. FedGraphNN [12][3] serves as a federated benchmark for undirected GNNs, supporting distributed training across non-IID domains via algorithms like FedAvg [24] and FedOPT [1], yet overlooks directed structures and synthetic graph synthesis. Additional distributed GNN works, such as DistGNN [25], which partitions graphs for parallel training on clusters, and PipeGCN [33], which pipelines aggregation to reduce latency, provide scalable training but lack distillation components for compression, making Dir-GD’s integration unique. Dir-GD uniquely supports directed graphs via distributed subgraphs, achieving up to 99.9% compression through community-driven partitioning, PageRank-based [9] node selection, and learnable parameter fusion with DAGCN-First [37], addressing the limitations

of prior works in scalability and directionality preservation while opening avenues for hybrid federated-distributed systems.

B Distributed Training Algorithm

Algorithm 4 Distributed Training

Require: Community C_i , data $(X, \text{edge_index}, Y)$, global parameters θ_g
Ensure: Local parameters θ_i

- 1: Extract subgraph (X_i, E_i, Y_i) from C_i
- 2: Create node mapping to reindex E_i
- 3: Initialize DAGCN model M_i with θ_g
- 4: Set train mask for all community nodes
- 5: **for** epoch = 1 to max_epochs **do**
- 6: Compute $\hat{Y}_i = M_i(X_i, E_i)$
- 7: Compute NLL loss $\mathcal{L}_i$ on train nodes
- 8: Update θ_i via optimizer
- 9: **if** early stopping condition met **then**
- 10: break
- 11: **end if**
- 12: **end for**
- 13: **return** θ_i

C Testing Algorithm

Algorithm 5 Testing

Require: Global model M_g , graph data $(X, \text{edge_index}, Y)$, test mask test_mask, batch size B
Ensure: Test accuracy acc

- 1: Initialize correct = 0, $N_{\text{test}} = \sum \text{test_mask}$
- 2: Set M_g to eval mode
- 3: **for** batch index = 0 to $\lceil |V|/B \rceil - 1$ **do**
- 4: Extract batch nodes V_b
- 5: Get $X_b, Y_b, \text{test_mask}_b$
- 6: **if** $\sum \text{test_mask}_b = 0$ **then**
- 7: continue
- 8: **end if**
- 9: Extract and remap batch edges E'_b
- 10: Compute $\hat{Y}_b = M_g(X_b, E'_b)$
- 11: Get predictions Y_b^{pred}
- 12: Update correct += $\sum \text{test_mask}_b[i] \cdot \mathbb{1}(Y_b^{\text{pred}}[i] = Y_b[i])$
- 13: **end for**
- 14: acc = correct/ N_{test}
- 15: **return** acc
